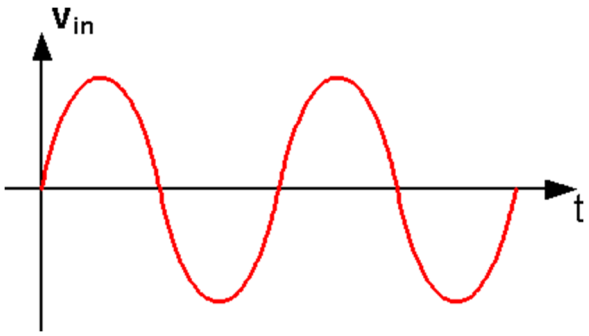
SC2107/CE2107 Microprocessor System Design and Development

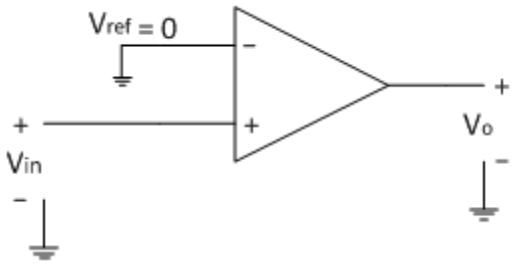
Tutorial 5 (with Solutions)

Sensor and Signal Conditioning, Analog Interface

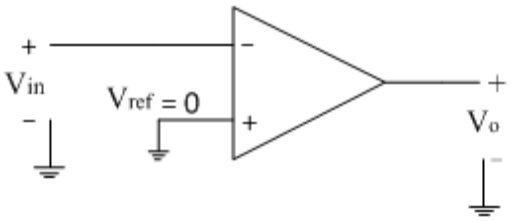
1. Plot the voltage vs. time of a non-inverting comparator and an inverting comparator for the following input voltage. The reference voltage for both comparators is 0V.



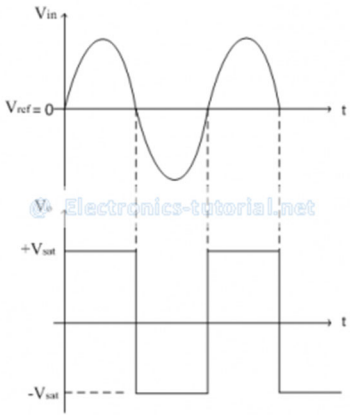
Solution:

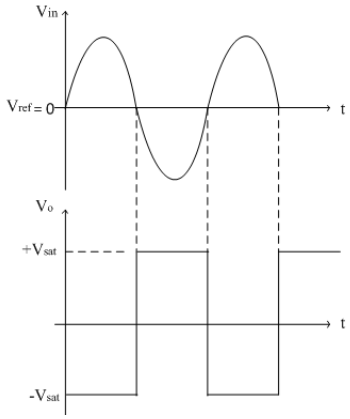


Non-Inverting Comparator

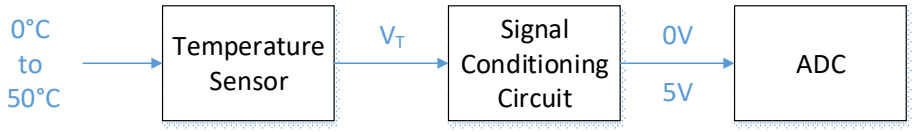


Inverting Comparator



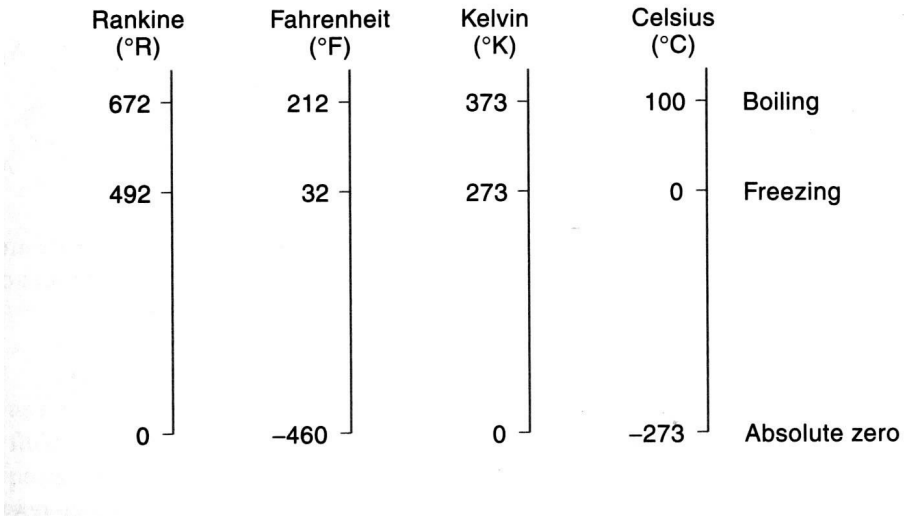


2. You are required to design a signal conditioning circuit to interface between a temperature sensor and the ADC. The temperature range to be measured is 0°C to 50°C. The range of the ADC converter is 0 to 5V.



The output of the signal conditioning circuit should be linear; i.e. when the sensor is measuring 0°C, the output of the signal conditioning circuit should be 0V; when the sensor is measuring 25°C, the output of the signal conditioning circuit should be 2.5V; when the sensor is measuring 50°C, the output of the signal conditioning circuit should be 5V.

The temperature sensor has a sensitivity of 10mV/°K. The relationship between degrees Kelvin and Celsius is: 1-degree rise in Kelvin equals a 1-degree rise in Celsius. The freezing point of water is 0°C, which equals to 273°K.



a. Write an equation that describes the input-output characteristic of the temperature sensor (in terms of V_T and the measured temperature in Celsius, T_C).

Solution

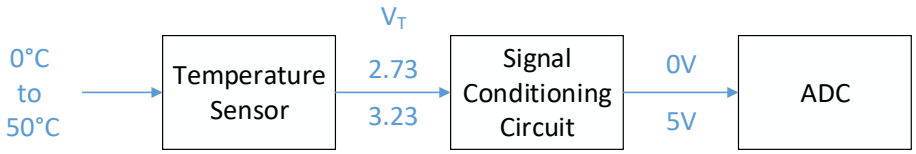
$$V_T = (273 + Tc) \times (10 \times 10^{-3})$$

b. Find the output voltage range of the temperature sensor (V_T).

Solution

The temperature sensor has a sensitivity of 10mV/°K and the freezing point of water is 273°K (0°C).

When measured temperature = 0°C, $V_T = (273 + 0) \times (10 \times 10^{-3}) = 2.73V$
When measured temperature = 50°C, $V_T = (273 + 50) \times (10 \times 10^{-3}) = 3.23V$

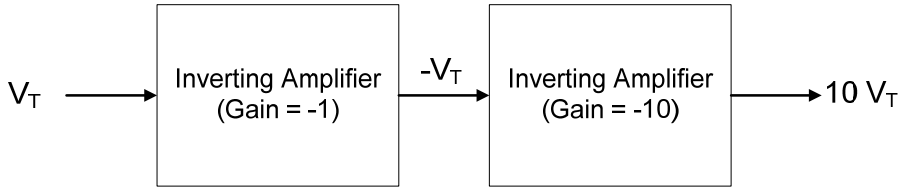


c. Find the gain that is required to amplify V_T to fit the input range of the ADC. Using inverting amplifiers, draw the block diagram of the signal conditioning circuit to realize this gain.

Solution:

$$Gain = \frac{5 - 0}{3.23 - 2.73} = 10$$

Since V_{out} should be positive, we can add another inverting amplifier with a gain of -1 to generate $-V_T$.



d. Explain why the circuit is not complete. Write a linear equation for the signal conditioning circuit output voltage to satisfy the requirements of the application.

Solution:

The circuit is not complete as we still need to map the range of V_T (2.73V – 3.23V) to the voltage range of the ADC (0V – 5V).

$V_{out} = (10 \times V_T) + V_{offset}$

Since the output of the signal conditioning circuit should be 0V when V_T = 2.73V,

$0 = (10 \times 2.73) + V_{offset}$

$V_{offset} = -27.3$

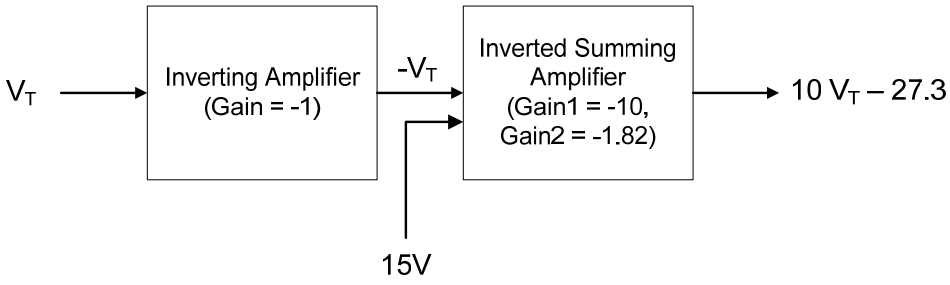
Hence, the linear equation for the signal conditioning circuit output voltage is:

$V_{out} = 10V_T - 27.3$

e. Briefly explain how you can modify the solution from 1(c) to implement the final circuit. Draw a block diagram of the final circuit.

Solution:

Replace the second inverting amplifier with an inverted summing amplifier. The first input of the inverted summing amplifier is $-V_T$ and a gain of -10 is applied to this input. The second input of the inverted amplifier could be 15V and a gain of -1.82 (-27.3/15) is applied to this input.



f. If a 3-bit ADC is used and the output generated by the ADC is 101_2, determine the measured temperature in Celsius T_C.

Solution:

Quantization size for ADC = $FS/2^N = (5-0)/2^3 = 0.625V$.

We can determine V_T (output voltage of the temperature sensor) using the equation of the signal conditioning circuit that was calculated in 1(d). Since, the ADC output is 101_2 or 5_{10},

$10V_T - 27.3 = 5 \times 0.625 = 3.125V$

$V_T = (3.125 + 27.3)/10 = 3.0425V$

Finally, we can use the equation of the sensor calculated in 1(a) to find T_C.

$T_C = 3.0425/(10 \times 10^{-3}) - 273 = 31.25°C$

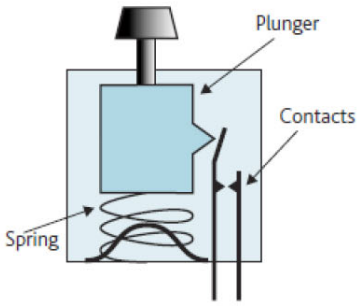
3. Answer the following questions on the hardware successive approximation ADC and digital ramp ADC.
   a. State the benefit of the successive approximation hardware converter over digital ramp ADC.
   b. Explain how this benefit is achieved by describing the working principle of the successive approximation converter.
   c. For an m-bit analogue-to-digital conversion, state the maximum number of iterations required for both the successive approximation and digital ramp ADC.

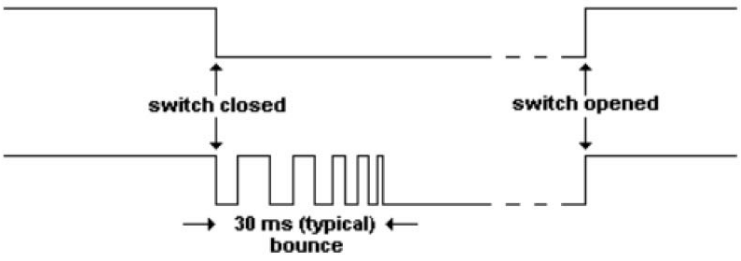
Solution:

a. The successive approximation converter has a shorter conversion time than the digital ramp ADC.
b. This is achieved by using a special counter circuit known as a successive-approximation register (SAR). Instead of counting up in binary sequence, the SAR counts by trying all values of bits starting with the most significant bit and finishing at the least-significant bit. Throughout the count process, the register monitors the comparator's output to see if the binary count is less than or greater than the analogue signal input, adjusting the bit values accordingly. This allows the successive approximation converter to converge on the analogue signal input in much larger steps than the 0-to-full count sequence of a regular counter in the digital ramp ADC.
c. Successive approximation – m iterations;
   Digital ramp – 2^m iterations

Optional

4. Switches suffer from Switch Bounce, which produces a series of pulses when contact is made. This can be registered as a series of ON and OFF signals lasting several milliseconds instead of just the one intended single and positive switching action. The easiest software approach for debouncing is Wait and See (see pseudo code below).





```
Set Return_Value = !Switch_Pressed

If (Switch_Pin == 0) // Switch is pressed
{
      WAIT (DEBOUNCE_PERIOD);
      If (Switch_Pin == 0) // If switch is still pressed
      {
            while (Switch_pin == 0);
            Return_Value = Switch_Pressed;
      }
}
return (Return_Value);
```

a) In certain applications, a single keypress is registered only upon the release of the button. How would you modify the pseudo-code of Wait and See above, to register the key press only when the button has been released?

b) Discuss a practical problem of your solution in a.

SC2107/CE2107 Microprocessor System Design and Development

Tutorial 6 (with Solutions)

Serial Bus and Display

1. Discuss the common and different characteristics of SPI and I2C?

Solution:

Common characteristics:

- Serial (single data line)
  - Synchronous
  - Data is clocked along with a clock signal
  - Master-Slave protocol
- Master device controls the clock (SCLK/SCL)
- Usually one master at a time

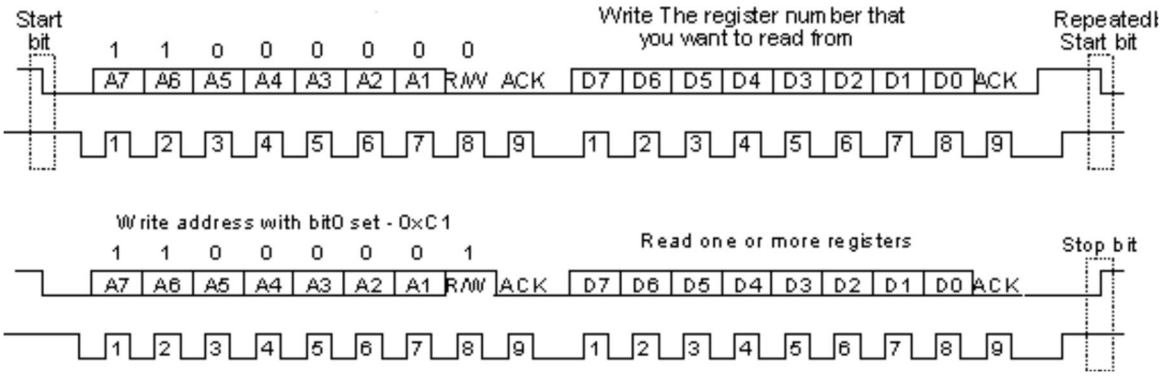
Different characteristics:

- SPI requires 3 or 4 wires, while I2C requires 2 wires (power and ground not considered).
- SPI has higher throughput due to full duplex. I2C is slower due to half duplex.
- In SPI, slave is selected using CS# (increase wiring). In I2C, slave is selected using 7-bit address.

2. In the I2C protocol, in order to read from the slave device, you must first tell it which of its internal addresses you want to read from. So a read of the slave actually starts off by writing to it. Let's look at the sequence of communicating with a peripheral module.
   - Send the Start Sequence
   - Send 0xC0 (I2C address of the peripheral module with R/W set to low)
   - Send 0x01 (Internal address of the register that we want to read from)
   - Send a Start Sequence again
   - Send 0xC1 (I2C address of the peripheral module with R/W set to high)
   - Read a byte from the peripheral device
   - Send the Stop Sequence

   Illustrate the SCL and SDA lines of the I2C bus to reflect the above sequence.

   Solution:



3. In the I2C protocol, there could be a situation where the slave is not ready to send data.

   Why is this a problem and describe the solution in the I2C protocol to overcome this problem.

Solution

Clock Stretching. The slave is allowed to hold the SCL line low. When the slave gets the read command from the master it holds the clock line low. The slave then gets the requested data, places it in the transmission register and releases the clock line allowing the pull-up resistor to finally pull it high. From the masters point of view, it will issue the first clock pulse of the read by making SCL high and then check to see if it really has gone high. If it's still low then the slave that holding it low and the master should wait until it goes high before continuing.

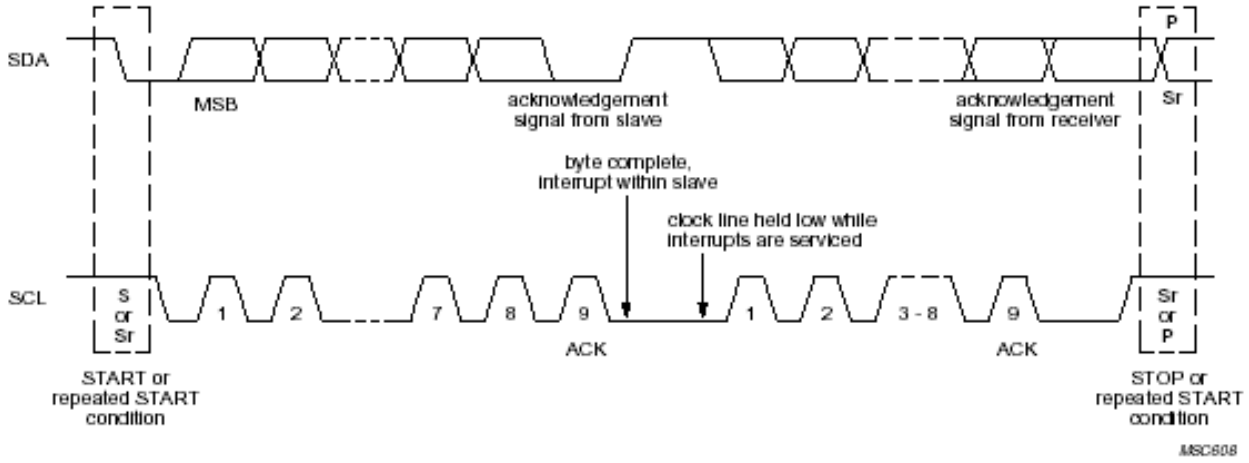
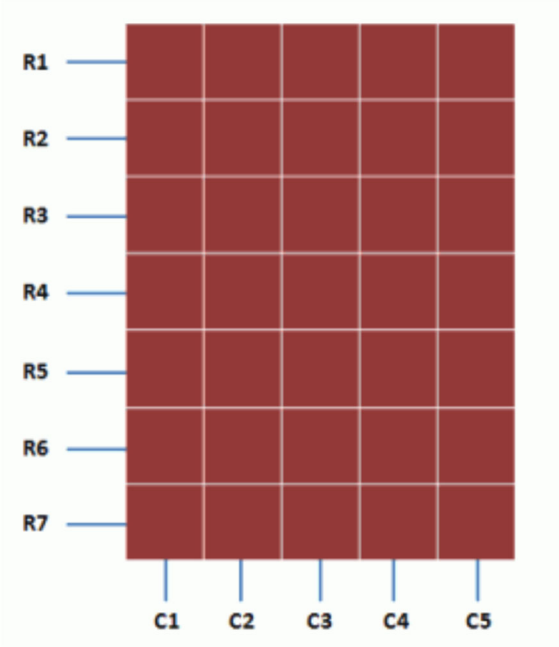


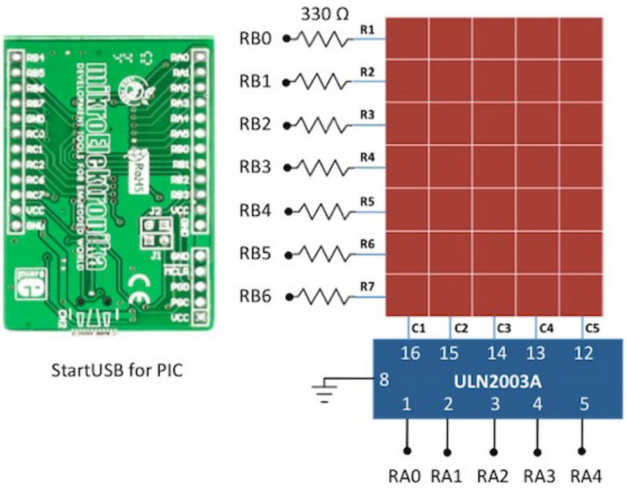
Fig.6 Data transfer on the I²C-bus.

4. A matrix LED as shown in the figure below is to be interfaced to a microprocessor. Explain how the interfacing will be done and the factors that should be considered.



Solution:

All the rows and columns need to be interfaced with GPIO lines of the microprocessor.



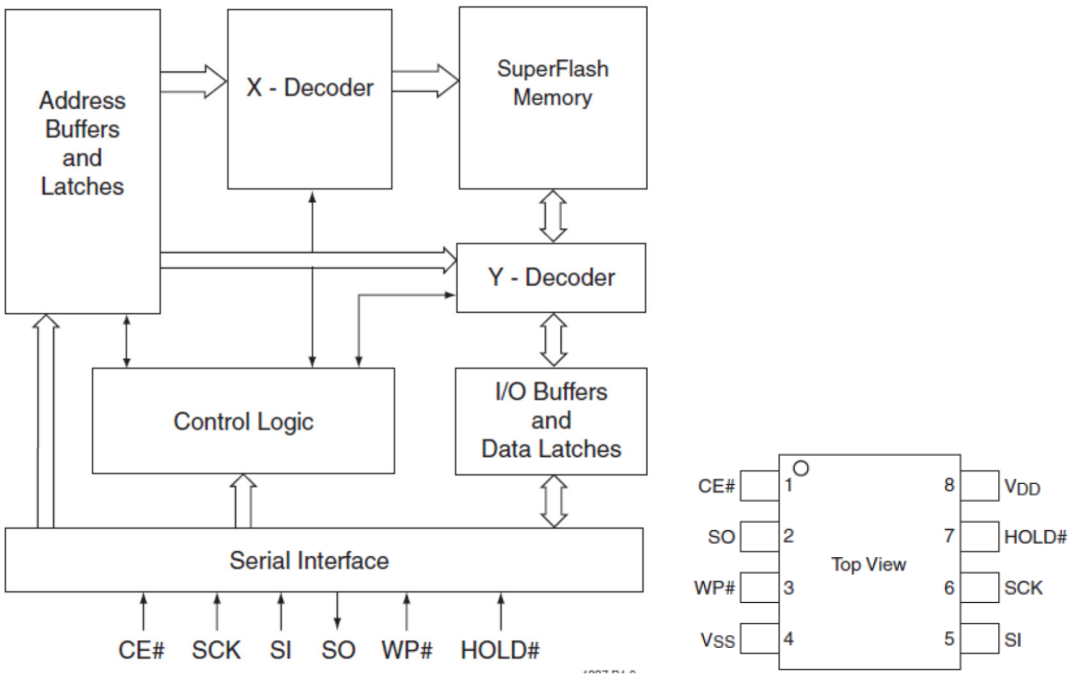
For the Rows, they can be interfaced through a current-limiting resistor. It is important to know the current-handling capability of the LEDs so as to ensure that the appropriate resistor value is chosen. This will also help in selecting the appropriate power rating for the resistors.

For the Columns, they need to be interfaced through a current-sinking transistor array. This is important because when multiple rows are biased through a single column, the current flowing through the column pin will be very high. This can potentially damage the microprocessor pin.

Optional

5. The SPI is a common interface for a microprocessor to communicate with peripheral chips. In order to communicate effectively, the programmer must have a good understanding of the protocols associated with that particular chip. These protocols may be unique to that particular chip or may be generalized due to an industry standard.

   We will look at some of the protocols of the 32Mbit SPI Flash chip (SST25VF032B) to have a better understanding.

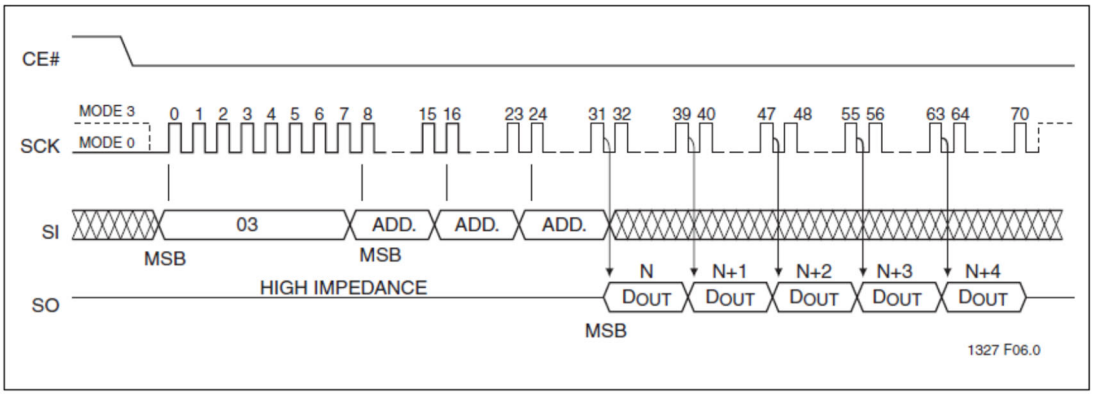


Block Diagram and Pin-Out of the SPI Flash Chip

The Device Operation Protocol is as follows:

Instruction	Description	Op Code Cycle[1]	Address Cycle(s)[2]	Dummy Cycle(s)	Data Cycle(s)	Maximum Frequency
Read	Read Memory	0000 0011b (03H)	3	0	1 to ∞	25 MHz
High-Speed Read	Read Memory at higher speed	0000 1011b (0BH)	3	1	1 to ∞	80 MHz
4 KByte Sector-Erase[3]	Erase 4 KByte of memory array	0010 0000b (20H)	3	0	0	80 MHz
32 KByte Block-Erase[4]	Erase 32KByte block of memory array	0101 0010b (52H)	3	0	0	80 MHz
64 KByte Block-Erase[5]	Erase 64 KByte block of memory array	1101 1000b (D8H)	3	0	0	80 MHz
Chip-Erase	Erase Full Memory Array	0110 0000b (60H) or 1100 0111b (C7H)	0	0	0	80 MHz
Byte-Program	To Program One Data Byte	0000 0010b (02H)	3	0	1	80 MHz
AAI-Word-Program[6]	Auto Address Increment Programming	1010 1101b (ADH)	3	0	2 to ∞	80 MHz
RDSR[7]	Read-Status-Register	0000 0101b (05H)	0	0	1 to ∞	80 MHz
EWSR	Enable-Write-Status-Register	0101 0000b (50H)	0	0	0	80 MHz
WRSR	Write-Status-Register	0000 0001b (01H)	0	0	1	80 MHz
WREN	Write-Enable	0000 0110b (06H)	0	0	0	80 MHz
WRDI	Write-Disable	0000 0100b (04H)	0	0	0	80 MHz
RDID[8]	Read-ID	1001 0000b (90H) or 1010 1011b (ABH)	3	0	1 to ∞	80 MHz
JEDEC-ID	JEDEC ID read	1001 1111b (9FH)	0	0	3 to ∞	80 MHz

The following figure shows the waveform for the ‘Read Sequence’ command.

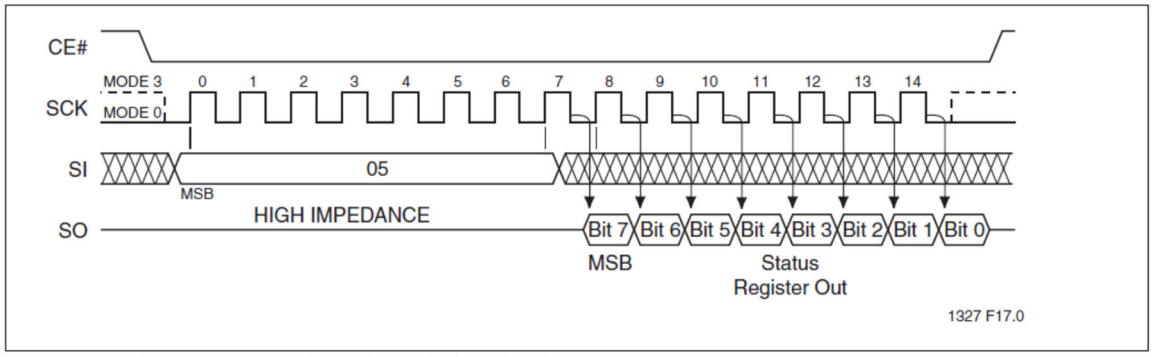


a. Decode the protocol sequence from the microprocessor's perspective.

Solution:

- The microprocessor must first select the device by driving its CE# low.
- The READ command (0x3h) is written to SI, followed by a 3-byte address (A23 – A0) that can be at any location in the device.
- The data at that address is available at the SO line.
- The device automatically increments to the next higher address after each byte of data is output.
- The data output can continue indefinitely.
- The READ command is terminated by driving CE# high at any time

The following figure shows another SPI transaction with the device.



b. Decode the protocol command.

Solution:

The command is encoded as 0x05 and according to the protocol table, it refers to Read-Status-Register (RDSR).

c. Decode the protocol sequence from the microprocessor's perspective.

Solution:

- The microprocessor must first select the device by driving its CE# low.
- The RDSR command (0x5h) is written to SI.
- The status register information is clocked out as the next byte.

Semiconductor Memory

1. The two tables below show a sample of commands available when interfacing to NAND flash (Table 1) and NOR flash (Table 2). What can you deduce from the nature of the Read/Program commands supported by the NAND and NOR Flash?

Table 1: NAND Flash Commands

Operation	Cycle 1	Cycle 2	Valid During Busy
PAGE READ	00h	30h	No
PAGE READ CACHE MODE START[1]	31h	–	No
PAGE READ CACHE MODE START LAST[1]	3Fh	–	No
READ for INTERNAL DATA MOVE[2]	00h	35h	No
RANDOM DATA READ[3]	05h	E0h	No
READ ID	90h	–	No
READ STATUS	70h	–	Yes
PROGRAM PAGE	80h	10h	No
PROGRAM PAGE CACHE[1]	80h	15h	No
PROGRAM for INTERNAL DATA MOVE[2]	85h	10h	No
RANDOM DATA INPUT for PROGRAM [4]	85h	–	No
BLOCK ERASE	60h	D0h	No
RESET	FFh	–	Yes

Table 2: NOR Flash Commands

		WRITE Operations											
		1st		2nd		3rd		4th		5th		6th	
Command	Length	Addr	Data	Addr	Data	Addr	Data	Addr	Data	Addr	Data	Addr	Data
READ/RESET	1	X	F0										
	3	555	AA	2AA	55	X	F0						
AUTO SELECT	3	555	AA	2AA	55	555	90						
PROGRAM	4	555	AA	2AA	55	555	A0	PA	PD				
UNLOCK BYPASS	3	555	AA	2AA	55	555	20						
UNLOCK BYPASS PROGRAM	2	X	A0	PA	PD								
UNLOCK BYPASS RESET	2	X	90	X	00								
CHIP ERASE	6	555	AA	2AA	55	555	80	555	AA	2AA	55	555	10
BLOCK ERASE	6+	555	AA	2AA	55	555	80	555	AA	2AA	55	BA	30
ERASE SUSPEND	1	X	B0										
ERASE RESUME	1	X	30										
READ CFI QUERY	1	55	98										

- NAND Flash supports PAGE READ and WRITE while NOR flash support Word Read/Write.
- The 'Program' command writes a word into a random address supplied. NOR flash architecture has better support for random read as it has sufficient address lines to map the entire memory.

- NAND flash requires multiplexed address and data sequence to enable a read, resulting in slower initial read. To compensate for this, read access is typically done in bulk at Page level.
- NAND flash typically has a cached version of the Page Read/Write, this helps to improve its performance. Important feature for storage. The cached Page Read also help to mitigate the slower Read operation of NAND flash memories.
- In general, NOR has a faster Random Read while NAND tend to do more bulk (Page level) read/write. This coupled with lower cost per bit makes NAND more suitable to be used as bulk storage memory while NOR is more suitable for system memory which requires random access.

2. The SSD controller manage the translation between Logical Block Address (LBA) from the Application processor to the Physical Block Address (PBA) of the actual storage location of the data in the SSD.
   - Table 3.1 shows the mapping of LBA to PBA.
   - For simplicity's sake, we can assume that these addresses at block level address, e.g. LBA block 0 (LBA0) is mapped to PBA block 9 (PBA9) in the table.
   - Table 3.2 shows the erase count of each PBA block.
   - LBA, PBA block correspond to the flash block in the flash structure, i.e. erasure is at block level.
   - A 'Y' in the 'Empty' column indicates that the Flash Block is empty.

(a) Complete Table 3.1 for Dynamic wear leveling algorithm. You can further assume that
   - Access sequence from the processor is LBA 7, 4, 3, A.
   - All access will go through the wear levelling algorithm for physical block assignment.
   - If the LBA was originally assigned to a different PBA, the PBA containing the old data will be erased.

   What is the consideration to take for the erasure process of the old data?

Table 3.1

LBA	PBA
0	9
1	X
2	X
3	5
4	X
5	4
6	6
7	7

Table 3.2

Erase Count Table

PBA	Erase Count	Empty
0	5	N
1	33	Y
2	15	Y
3	98	Y
4	45	N
5	11	N
6	69	N
7	78	N

8	X
9	X
A	0

8	A3	Y
9	F1	N
A	56	Y

- Evaluation of which PBA block to assign is done before any PBA block erasure. New data must be re-written to the new target PBA block before erasing the old data.

Dynamic

Table 3.1	
LBA	PBA
0	9
1	X
2	X
3	5 → A(3)
4	X → 1(2)
5	4
6	6
7	7 → 2(1),
8	X
9	X
A	0 → 5(4)

Table 3.2 (Dynamic)		
Erase Count Table		
PBA	Erase Count	Empty
0	5 → 6(4)	N → Y(4)
1	33	Y → N(2)
2	15	Y → N(1)
3	98	Y
4	45	N
5	11 → 12(3)	N → Y(3) → N(4)
6	3	N
7	78 → 79(1)	N → Y(1)
8	A3	Y
9	F1	N
A	56	Y → N(3)

(b) In table 3.2, you can see that PBA block 6 has a very low erase count but would not be evaluated under Dynamic wear levelling algorithm. Static wear levelling attempts to relocate such flash blocks that are seldom re-written to achieve more even erase count distribution. There are many ways to implement Static wear levelling, below is one example. Go through the algorithm described and suggest enhancement to the algorithm.

- On every access, all blocks (occupied and empty) will be evaluated for block assignment.
- PBA block with the lowest erase count will be selected to store the data.

Enhancement Suggestions

- In the current method, if the chosen block to write into is already occupied, the target block will have to be erase and relocated, this will result in more erasure operation, increasing the total erase counts. To avoid this issue, separate the evaluation of occupied and empty flash blocks
    a. Evaluate empty blocks to decide which flash blocks to use for active re-writing of blocks.
    b. Evaluate occupied blocks periodically to re-locate occupied blocks with low erase counts, freeing these blocks for usage for subsequent access.
- Occupied blocks in part (b) above are only relocated to empty blocks to avoid unnecessary erasure process.

- Empty blocks with high erase counts can be chosen to store data which are seldom re-written into. May or may not be possible as this will need info on LBA access statistics, and provided the files associated with the LBA stays constant.

3. Figure 5.2a and 5.2b shows the Read and Write timing diagram of a processor's external memory interface. The processor can configure the access timing via three parameters: setup, strobe and hold. The description of these parameters can be found in Figure 5.1. The example in the figure is configured for setup = 2, strobe = 5 and hold =1. Figure 5.3a and 5.3b shows the read and write access timing diagram of a SRAM. Configure the processor's external memory interface to match SRAM timing. Following are some information that you'll need for the computation
    - CPU Clock cycle = 5 ns
    - The unit of the setup-strobe-hold parameters are in CPU clock cycles.
    - Processor ARE_ pin is connected to SRAM OE_ pin
    - Processor latch in the data on rising edge of ARE_
    - Maximum value of SRAM tOE = 40 ns
        - tOE is the Output Enable to Output Valid timing.
    - Minimum value of SRAM tWP = 40ns
        - tWP is the Write Pulse Width timing
    - Minimum tDH = 5 ns
        - tDH is the Data Hold from WE signal inactive
    - Minimum tAS = 0 ns.
        - tAS is the Address setup time before WE signal is active

    - Max tOE_ = 40ns and min tWP = 40ns. Strobe = 40/5 = 8.

    - Min tDH = 5ns => hold = 5/5 = 1

    - Setup is less critical. Can be setup = 1 since minimum tAS = 0.

Parameters	Control Bits	Definition
Setup periods	RDSETUP WRSETUP	A setup period is the time in CPU clock cycles given to setup the address, chip enable ($\overline{CE}$), and byte enable ($\overline{BE}$) signals before the read strobe signal ($\overline{ARE}$) or write strobe signal ($\overline{AWE}$) falls. For an asynchronous read operation, this is also the setup period for the output enable signal ($\overline{AOE}$) before $\overline{ARE}$ falls.
Strobe periods	RDSTROBE WRSTROBE	A strobe period is the time in CPU clock cycles between the falling (activation) and rising (deactivation) of the read or write strobe signal.
Hold periods	RDHOLD WRHOLD	A hold period is the time in CPU clock cycles during which the address and byte enable lines are held active after the read or write strobe signal rises. For an asynchronous read operation, this is also the hold period for the output enable signal after $\overline{ARE}$ rises.

Figure 5.1: External Memory Interface Configuration Parameters

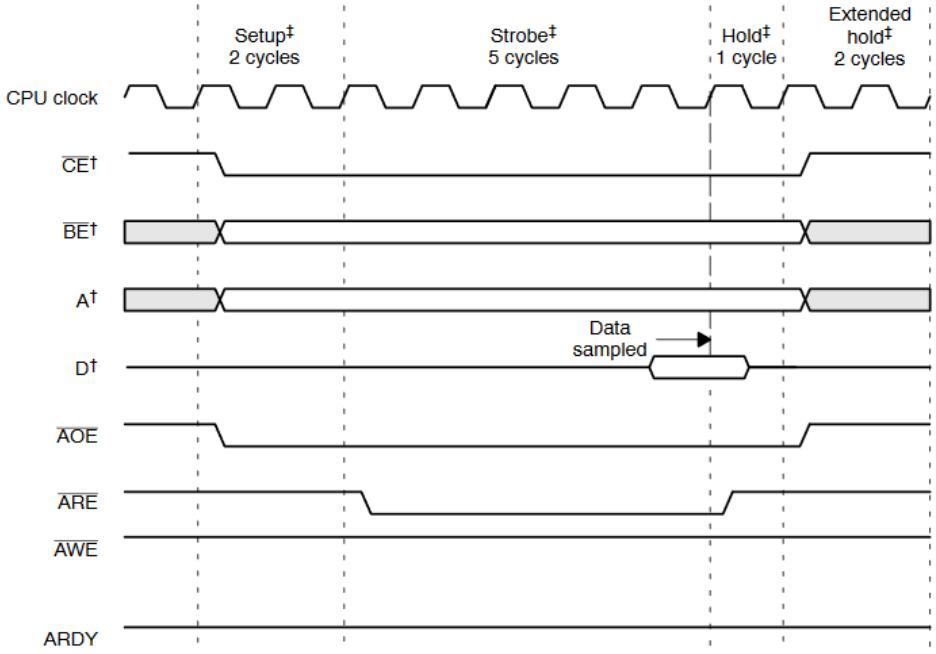


Figure 5.2: Processor External Read Timing Diagram

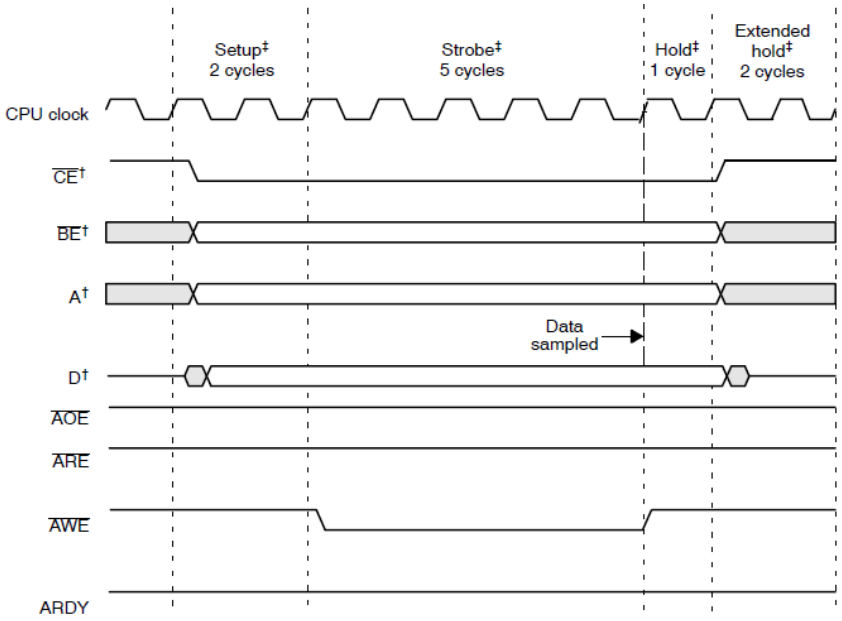


Figure 5.2b: Processor External Write Timing Diagram

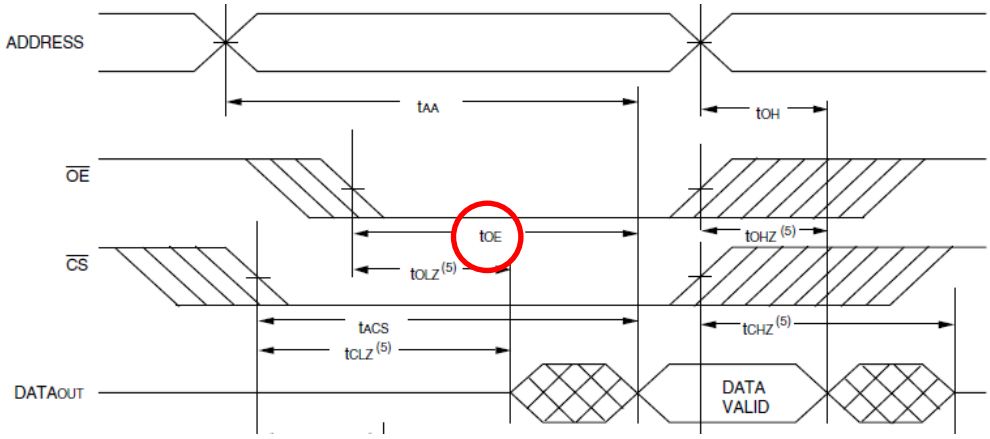


Figure 5.3: SRAM Read access timing diagram

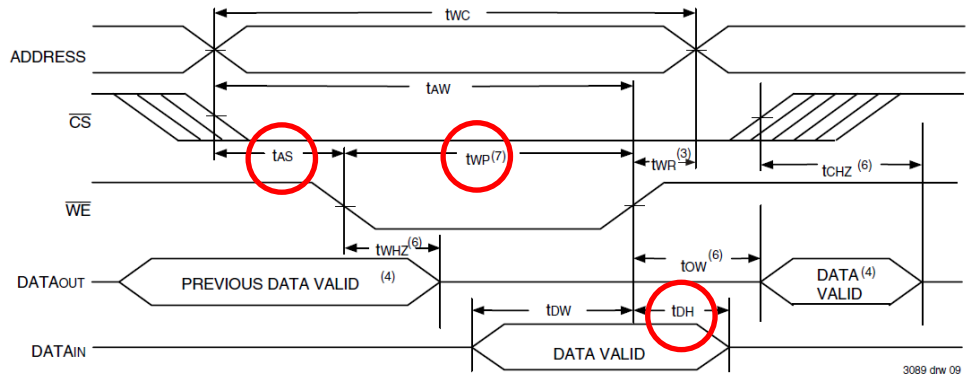
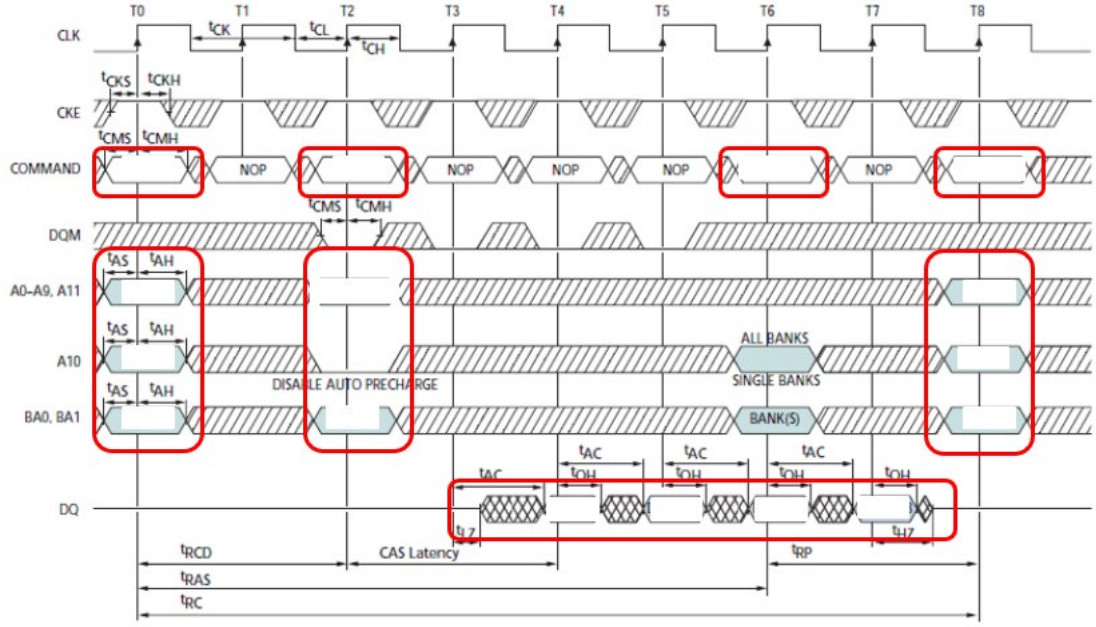


Figure 5.3b: SRAM Write access timing diagram

4. Complete the timing diagrams in Table 6.1 and 6.2 below, fill in the missing information on commands, data, addresses etc.
   - What is the CAS latency of the SDRAM for Table 6.1?
   - Table 6.1: CAS Latency =2.

Table 6.1: SDRAM Read without Auto-Precharge



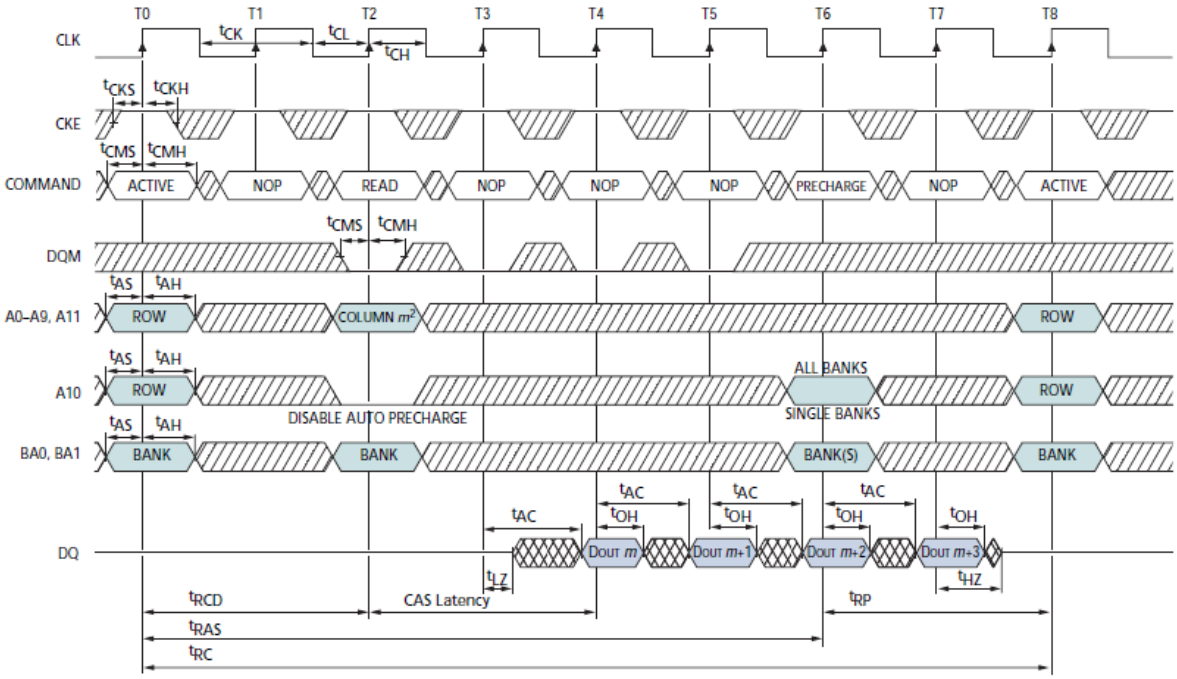
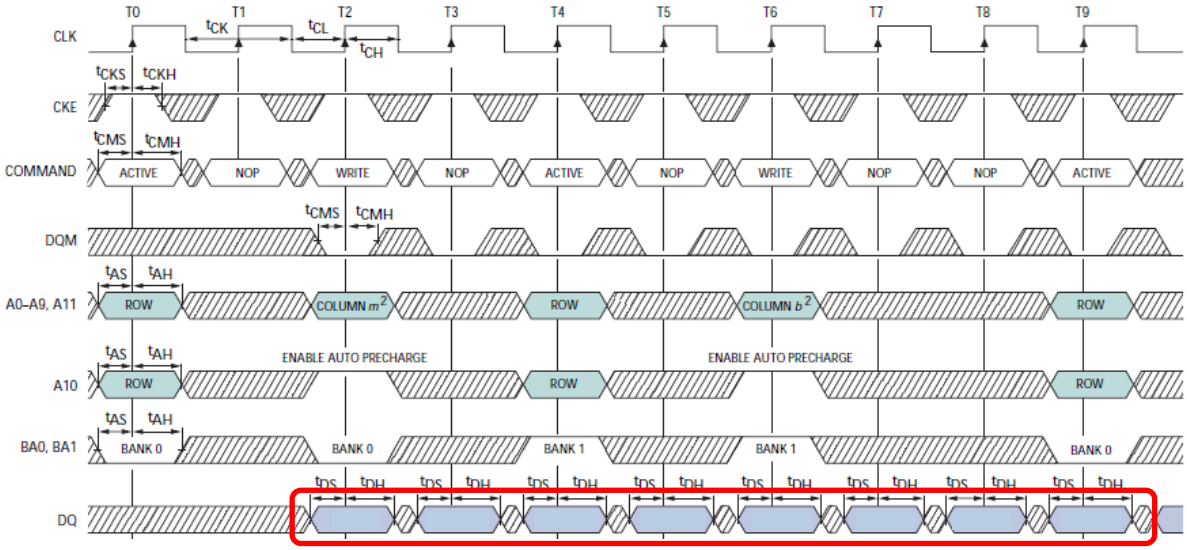
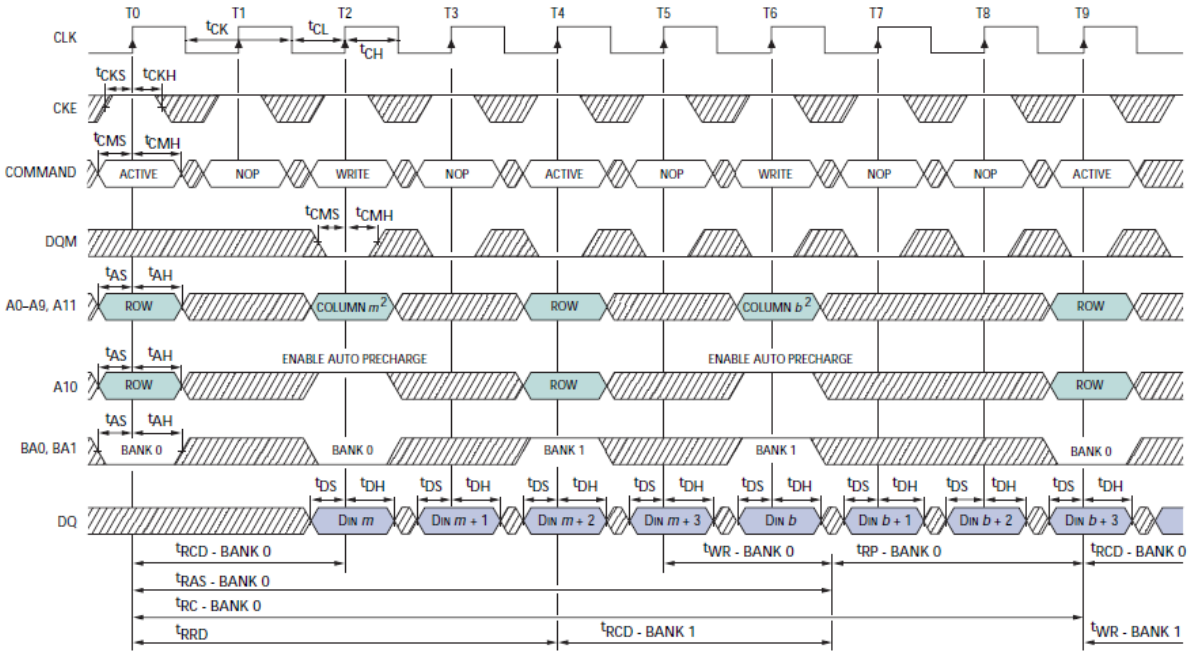


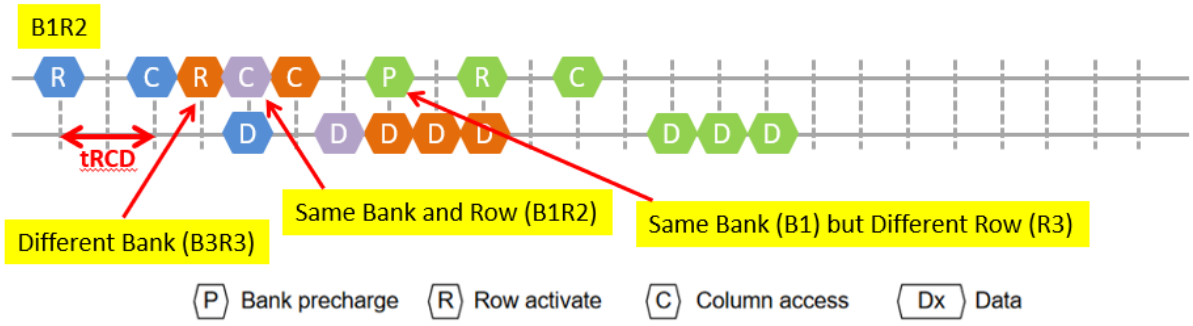
Table 6.2: SDRAM Write – Alternating Bank Access (with Auto-Precharge)





5. Arrange the access sequence to optimise the access time for the following data. Given that the tRCD(min) = 2 SDRAM clocks. tRCD is the minimum delay needed after an ACTIVE command before user can issue the READ/WRITE command to the SDRAM.
    - Bank1-Row2-Byte0 (1 byte)
    - Bank3-Row3-Byte2 (3 bytes)
    - Bank1-Row3-Byte2 (3 bytes)
    - Bank1-Row2-Byte3 (1 byte)

- Second access (B3R3B2) was brought forward as it is from a different bank, so access can be overlapped with the first access (B1R2B0).
- Fourth access (B1R2B3) is brought forward as well to save on one pre-charge cycle.



(Optional)

6. SSD Design considerations: homework given during lecture

Over-provision

- over-provisioning refers to a function that secures extra space to allow for efficient use of the SSD by allo-cating a certain amount of the SSD's NAND flash to an over-provisioning space. This space can only be accessed by the SSD's con-troller and not by the host. Consisting of free blocks only, the OP region assists in efficient delivery of free blocks when wear-lev-eling or garbage collection is in progress and contributes to improved performance and lifetime of the SSD.
- Although there is no difference between the sequential and random write performance for fresh-out-of-the-box (FOB) NAND, the random write does not perform as well as the sequential write once data has been written over the entire space of the NAND. Random writes, smaller in size than sequential writes, mix valid and invalid pages within blocks, which causes frequent GC and results in decreased performance. If the OP is increased, more free space that is inaccessible by the host can be secured, and the resulting efficiency of GC contributes to improved performance. The sustained performance is improved in the same manner.

CE2107 – OHL 1

SSD Design Consideration

- **Garbage Collection**
- Technique used to free up previously written blocks
- Consolidates pages by moving and rewriting pages from multiple blocks to fill up fewer new ones.
- Old blocks are then erased to provide storage space for new incoming data.

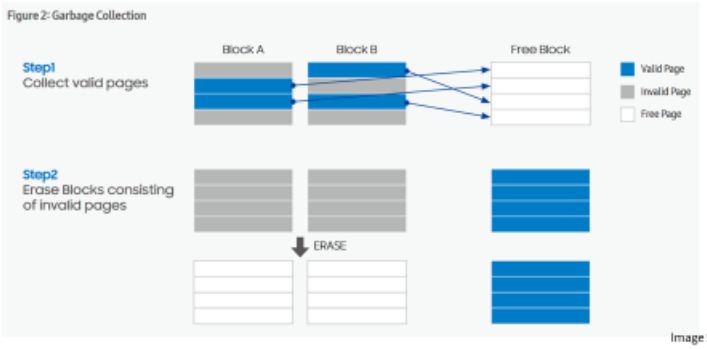


Image Source: Samsung

CE2107 – OHL 43

SSD Design Consideration

- **Write Amplification**
- Smallest unit that can be programmed is a Page
- Smallest unit that can be erased is a Block (Multiple Pages)
- To overwrite a location in Flash, it has to be erased first(unlike HDD)
- As the drive is used, data changes, and the changed data is written to other pages in the block or to new blocks. At this point, the old (stale) pages are marked as invalid and can be reclaimed by erasing the entire block. To do this, however, any still-valid information on all of the other occupied pages in the block must be moved to another block.
- The requirement to relocate valid data and then erase blocks before writing new data into the same block causes **write amplification**;
- Write Amplification happens when the total number of writes required at the flash memory is higher than the host computer originally requested.
- It also causes the SSD to perform write operations at a slower rate when it is busy moving data from blocks that need to be erased while concurrently writing new data from the host computer.

CE2107 – OHL 44

SSD Design Consideration

- **TRIM/UMAP**
- The File Allocation Table maintained by the OS is not visible to the SSD, as such, when a file is deleted, the SSD do not know that the corresponding flash blocks can be freed. Only the storage corresponding to the FAT is modified.
- Certain Operating Systems support the TRIM/UNMAP function, which translates deleted files to the associated LBA (logical block address) on the storage device (SSD).
- The TRIM/UNMAP command notifies the drive it no longer needs data in certain LBAs (Logical Block Address) which then free up a number of NAND pages.
- The TRIM/UNMAP command needs to be supported by the OS, the drive, and the controller in order to work.
- The TRIM/UNMAP command could result in higher SSD performance from both the reduced data needed to be rewritten during garbage collection and the higher free space resulting on the drive.

CE2107 – OHL 45

SSD Design Consideration

- When Host OS delete a file, only the FAT is modified, The actual data is still intact
- The Flash Controller does not know that the deleted locations are 'invalid' from Host OS point of view.
- The controller will continue to perform Garbage Collection for these locations. Meaning unnecessary data movement which reduces reliability, endurance and write performance.
- **TRIM/UMAP** command can be sent by OS to the Flash Controller to inform it of blocks that are marked invalid so that these block can be excluded from the Garbage collection and assigned to the free pool.

CE2107 – OHL 46

Microprocessor System Design Considerations

SoC Technology Roadmap and Trend

1. A portable Audio/Video Media player is designed based on the specification given below.
    - Li-Polymer Battery.
    - Audio Playtime 24hr, Video Play-time 10hr, Voice Recording 10hr.
    - USB3 slave connectivity
    - Standby time 7 days.
    - 1920x1080 LCD with Capacitive Touch Screen (SPI interface)
    - 8 user input buttons
    - System wakeup from standby via any button and SPI activity within 10μs
    - Software
        - Main system overhead: 20MIPS (Million Instruction per second)
        - MP3 decoding: 15 MIPS
        - MP4 decoding: 50 MIPS
        - Voice recording: 5 MIPS
        - Total program code size: 240KByte
        - Total program data size: 128Kbyte
        - Total data buffer size: 256MByte
    - Processor power consumption
        - System Overhead: 5mA/MIPS
        - Active (Audio/Voice functions): 5mA/MIPS
        - Active (Audio+Video functions): 10mA/MIPS
        - Standby: 3mA
    - Processor internal memory
        - 256 KByte SRAM, 512 KByte Flash

    (a) What external storage and system memory is suitable for this product? Or are they required?
        - Program Code of 240Kbyte < 512 KByte Flash
        - Program Data of 128Kbyte < 256 KByte RAM
        - => all can fit into the internal memory of the processor
        - 256Mbyte data buffer needs to be resided in external SDRAM, volatile memory chosen since large amount read/write operations is expected.
        - External NAND Flash used for storing Audio/Video media files. 'Portable' points to need for small form factor, hence NAND flash chosen for its size as well as cost/bit advantage.

(b) Does the processor require any special internal peripheral to interface to the memories listed above?

- SDRAM requires special controller logic to manage the various complex SDRAM commands/operation, and to enable code to be executed directly from there.
- NAND could use standard SRAM memory interface (with software drivers for NAND flash management) or a NAND Flash controller module.

(c) Calculate the power consumption when performing the following function and calculate the battery capacity required (in mAh) to meet the requirement specified. MP4 includes audio and video. Assume system overhead is the same for all activities. Ignore power consumed by other components on the player.

- Audio Decode
- 20*5 +15*5 = 175mA => 24*175 = 4200mAh
- MP4 Decode
- 20*5 + 50*10 = 600mA => 10*600 = 6000mAh
- Voice Record
- 20*5 +5*5 = 125mA => 10*125 = 1250mAh
- Standby (System Sleeping)
- 3mA*7*24 = 504mAh

Video recording require the most battery capacity. Hence a 6000mAh battery is required to meet the specifications.

(d) What could be done to reduce the battery capacity but still able to meet the various product function requirement?

- Put the processor to sleep whenever possible, e.g. after performing the audio/video decoding, voice recording/playback.

(e) What could be done if we want to increase the standby time?

- Reduce the standby power. E.g. Have a separate controller to manage the system during standby. Small MCU typically has smaller standby current, some can achieve down to uA standby.

(f) Audio codec typically requires a special clock frequency that is a multiple of the sampling frequency. Assuming that the sampling frequency, fs=44.1Khz, one of the common clock frequencies used is 256fs. How can this frequency be generated for this system? External Crystal used by the Processor has a frequency of 48Mhz.

- Not possible to generate multiples of 44.1Khz with a base frequency of 48Mhz. Will need an external clock generator or separate crystal/oscillator with frequency at multiple of 44.1Khz.

(g) Name one important reason why the processor needs a 48MHz crystal.

- USB controller needs a very accurate 48Mhz base clock transfer data over USB interface.

2. With reference to the information derived in Q1, some questions below

- Based on the very fast wakeup time requirement of 10µs, should a crystal be used during standby or an on-chip oscillator?
- What other points should be considered where pin connection and termination are concerned?

- The wakeup time when using external crystal should be tested to see if the 10us could be met. In general, crystal oscillation requires some time to stablise so an on-chip oscillator should be use or the external crystal oscillation shouldn't be shut down during standby (power consumption permit)
- Ensure all input pins are terminated to avoid excessive current drawn due to I/O pad's transistors being held at Vcc/2 level. Some pins have multiple functions mux'ed together, check that these functions are not used simultaneously.

3. Further to the system in Q1, what are the power supply design considerations given the following?
   - The processor selected has separate voltage rails for its core and I/O.
   - The system includes mixed signal components such as audio codec, video codec, ADC etc.
   - Any other additional power management considerations.
   - Check if there are any requirement on Processor Power supply sequencing, may need a separate power management IC.
   - Separate digital and analog voltage rails to isolate digital noise from the analog voltage supply. This will increase the accuracy of the ADCs and CODECs.
   - Apply Dynamic Voltage Frequency Scaling (DVFS), take note of the proper sequencing requirement of changing CPU frequency vs Core Voltage levels.

4. What is value proposition of the heterogeneous processing architecture that ARM proposed? How does ARM's big.LITTLE™ processor architecture managed to fulfill the these value proposition?

Value Proposition

- To use "just sufficient" resources to perform a particular function to achieve maximum power efficiency
- To be able to adapt dynamically and efficiently to changing application loading

"Just sufficient" resource deployment

- ARM big.LITTLE™ technology is a heterogeneous processing architecture which uses two types of processor core.
  - "LITTLE" processors are designed for maximum power efficiency
  - "big" processors are designed to provide maximum compute performance.
  - Processor vendor is allowed to mix and match different number of big and LITTLE cores to suit their targeted application's loading.
- When the application loading is light, the LITTLE processor will be used to save power. When the loading when beyond that of the LITTLE processor, the big processor will be switched in to carry the heavier load.

Dynamic and efficient adaption to application load

- One of the main issues with heterogeneous processing, or in fact, that of multi-processing are the data paths between processing cores. It is usually slower than the processor core speed so tends to be the bottleneck. Especially if one of the main features is to be able to dynamically move program around the system.
- ARM build a special interconnect known as the "Cache Coherent Interconnect" (CCI-400) enable the seamless data transfer between clusters, particularly between cache of each processors, without having to go through the relatively slower external memory
- Employment of Global Task Scheduling allows full flexibility to allocate a particular task to any available processor core.